

C++ 연습 문제: 06. 연산자 오버로딩과 사용자 정의 타입

1. 연산자 오버로딩에 대한 설명으로 옳바른 것은? (하)

1. 새로운 기호를 만들어 완전히 새로운 연산자로 사용할 수 있다.
2. 기본형(int, float 등) 타입들 사이의 연산자 동작을 임의로 변경할 수 있다.
3. 연산자의 우선순위나 피연산자의 개수를 변경할 수 없다.
4. 클래스의 멤버 함수로만 연산자를 오버로딩할 수 있다.

2. 다음 중 C++에서 오버로딩할 수 없는 연산자는 무엇인가? (하)

1. + (덧셈)
2. :: (범위 지정)
3. [] (배열 인덱스)
4. == (동등 비교)

3. 다음 빈칸에 들어갈 알맞은 낱말은? (중) [연산자 오버로딩 시 () 함수로 선언하면 클래스 외부의 전역 함수에서도 클래스의 **private** 멤버에 접근하여 연산자를 구현할 수 있다.]

1. **const**
2. **static**
3. **inline**
4. **friend**

4. 3차원 벡터(Vector3) 클래스의 덧셈 연산자(+)를 멤버 함수로 오버로딩할 때 가장 적절한 함수 시그니처(Signature)는 무엇인가? (중)

1. `void operator+(const Vector3& other);`
2. `Vector3 operator+(const Vector3& other) const;`
3. `Vector3& operator+(Vector3 other);`
4. `const Vector3& operator+(const Vector3& other);`

5. 문자열이나 동적 배열에서 인덱스를 통해 특정 원소에 접근하기 위해 오버로딩해야 하는 연산자는 무엇인가? (하)

1. `operator()`
2. `operator[]`
3. `operator->`
4. `operator*`

6. 배열 컨테이너(예: **FixedArray**)를 설계할 때, 일반 객체와 상수(**const**) 객체 모두에서 인덱스 접근이 가능하게 하려면 어떻게 해야 하는가? (상)

1. `operator[]` 하나만 **public**으로 열어두면 컴파일러가 알아서 처리한다.
2. **const** 한정자가 붙은 `operator[]`와 붙지 않은 `operator[]`를 각각 오버로딩한다.
3. **mutable** 키워드를 사용하여 변수를 선언한다.

4. 상수 객체에서는 인덱스 접근을 아예 불가능하게 막아야 한다.

7. 컨테이너 내부 원소 접근 시 범위를 벗어났는지 검사하는 기능과 관련하여 일반적인 표준 설계 관행으로 올바른 것은? (중)

1. `operator[]`는 성능을 위해 범위 검사를 생략하고, `At()` 함수는 범위 검사를 수행하여 예외를 던진다.
2. `operator[]`와 `At()` 모두 예외 없이 범위 검사를 생략해야 한다.
3. `operator[]`는 예외를 던지고, `At()`은 프로그램의 실행을 즉시 종료(`abort`)한다.
4. `operator[]`는 범위를 벗어나면 배열의 크기를 자동으로 늘려준다.

8. 객체가 유효한 상태인지 조건문(예: `if(handle)`)에서 확인하기 위해 명시적(`explicit`)으로 제공해야 하는 형 변환 연산자는 무엇인가? (상)

1. `operator int()`
2. `operator void*()`
3. `explicit operator bool()`
4. `explicit operator !()`

9. 단항 연산자인 전위 증가 연산자(`++x`)와 후위 증가 연산자(`x++`)를 오버로딩으로 구분하기 위한 문법적 특징은 무엇인가? (상)

1. 전위 증가 연산자는 매개변수로 `float`을 받는다.
2. 후위 증가 연산자는 구분을 위해 `int`형 매개변수를 하나 받는다.
3. 후위 증가 연산자는 함수 이름으로 `operator++`를 사용한다.
4. 전위 증가 연산자는 항상 `void`를 반환해야 한다.

10. 전위 증가 연산자(`++x`)를 오버로딩할 때 보통 권장되는 반환 타입은 무엇인가? (중)

1. 객체 자신의 복사본 (Value)
2. 객체 자신에 대한 참조 (Reference)
3. `void`
4. `bool`

11. 표준 입출력 스트림(`std::cout`)에서 사용자 정의 객체를 직접 출력(`cout << obj`)하기 위해 전역 함수로 오버로딩해야 하는 연산자는 무엇인가? (중)

1. `operator<<`
2. `operator>>`
3. `operator()`
4. `operator[]`

12. 연산자 오버로딩 시 피연산자의 조건에 대한 설명으로 올바른 것은? (중)

1. 피연산자 중 적어도 하나는 반드시 사용자 정의 타입(클래스나 열거형)이어야 한다.
2. 피연산자가 모두 기본형 타입이어도 마음대로 재정의할 수 있다.
3. 오버로딩된 연산자는 항상 두 개의 피연산자를 가져야 한다.

4. 포인터 타입들 사이의 덧셈 연산 동작을 오버로딩으로 바꿀 수 있다.

13. 다음 빈칸에 들어갈 알맞은 기호는? (하) [포인터처럼 동작하는 스마트 포인터나 반복자 객체를 만들 때, 객체가 가리키는 실제 데이터에 접근하기 위해 멤버 접근 연산자인 `->`와 역참조 연산자인 `()`를 오버로딩한다.]

1. `&`
2. `*`
3. `::`
4. `[]`

14. 어떤 클래스(예: `ResourceHandle`)에 `explicit operator bool() const;`가 정의되어 있을 때, 암시적 형 변환이 제한되어 컴파일 오류가 발생하는 코드는 무엇인가? (상)

1. `if (handle) { }`
2. `if (!handle) { }`
3. `bool b = handle;`
4. `while (handle) { }`

15. 객체를 마치 함수처럼 호출(예: `functor(x, y)`)할 수 있게 만들어 주는 연산자 오버로딩은 무엇인가? (하)

1. `operator()`
2. `operator[]`
3. `operator->`
4. `operator=`

16. 다음 중 연산자 오버로딩을 설계할 때 가장 바람직한 원칙은 무엇인가? (하)

1. 코드의 길이를 줄이기 위해 의미와 무관한 연산자를 다수 도입한다.
2. 원래 연산자가 가진 직관적인 의미(예: `+`는 결합, `==`는 동등성)와 일치하게 정의한다.
3. 보안 강화를 위해 덧셈(`+`) 기호를 내부적으로 뺄셈 동작으로 정의한다.
4. 메모리 낭비를 줄이기 위해 무조건 반환 타입을 `void`로 통일한다.

17. 이항 연산자인 덧셈(`+`)을 멤버 함수가 아닌 전역 함수(혹은 `friend`)로 오버로딩하는 가장 큰 이유는 무엇인가? (상)

1. 덧셈 연산은 전역 함수로만 구현해야 컴파일이 되기 때문이다.
2. 멤버 함수로 구현하면 `private` 변수에 접근할 수 없기 때문이다.
3. 왼쪽 피연산자가 기본형(예: `10 + obj`)일 때도 교환 법칙을 자연스럽게 지원하기 위해서다.
4. 메모리 누수를 원천적으로 방지하기 위해서다.

18. 산술 연산자(`+`)와 복합 대입 연산자(`+=`)의 관계에 대한 구현 관행으로 가장 올바른 것은? (중)

1. `+`를 먼저 구현하고, `+=`는 내부적으로 `+`를 호출하여 구현한다.
2. `+=`를 먼저 멤버 함수로 구현하고, 전역 함수 `+`는 복사된 객체에 `+=`를 적용하여 재사용한다.
3. 두 연산자는 연관성이 없으므로 각각 완전히 독립적으로 계산 로직을 작성해야 한다.
4. `+=` 연산자는 오버로딩이 불가능하다.

19. 다음 빈칸 (A)에 들어갈 알맞은 반환 타입은? (상) [대입 연산자(operator=)는 연속적인 대입(예: `a = b = c;`)을 지원하기 위해 보통 (A)을(를) 반환한다.]

1. `void`
2. 객체 자신에 대한 값 (Type)
3. 객체 자신에 대한 참조 (Type&)
4. `bool`

20. 다음 중 연산자 오버로딩에 해당하는 사례가 아닌 것은? (하)

1. `std::string` 타입의 두 문자열을 `+` 기호로 이어 붙인다.
2. `std::vector` 컨테이너의 원소에 `[]` 기호로 접근한다.
3. 객체의 멤버에 접근할 때 사용하는 `.` (점) 연산자를 오버로딩하여 다른 동작을 수행한다.
4. `std::cout` 객체에 `<<` 기호를 사용하여 다양한 변수 값을 출력한다.